

Constraint-Aware Random Lunch Recommendation in a Static Web Application

Student: Aleksandr Mandrov **Team:** Individual **Email:** apmandrov@edu.hse.ru
Date: 2026-09-15 **Assignment:** A01 - Random Lunch Generator (Week 1)

Abstract - A naive random lunch generator can repeat the same dish, overuse a food family, or recommend an explicitly rejected item. I extended a no-build HTML/CSS/JavaScript baseline with cluster-aware filtering, a three-recommendation cooldown, a session blacklist, balanced catalog coverage, valid icons, and low-inventory feedback. The final deterministic regression suite passed **13/13 tests**; three separate 100,000-selection checks reported zero identical-adjacent, same-cluster-adjacent, and cooldown violations while reaching all 42 dishes. The result shows that small, explicit candidate filters can substantially improve variety and user control without a backend.

Index Terms - recommender systems, random sampling, diversity, constraints, testing

1. Introduction

The baseline selected one of 12 hard-coded dishes directly from `Math.random()`. It had no memory, no preference controls, and three icon class names that did not exist in the pinned Font Awesome stylesheet. This is a small recommender, but its failures are recognizable recommendation problems: repetition, low catalog diversity, near-duplicate results, and inability to reject items.

Concrete scenario. With deterministic inputs 0.01 and 0.01, the baseline produced `Pizza` -> `Pizza`. A cluster-only revision could later produce `Pizza` -> `Sushi` -> `Pizza`, so preventing only immediate duplicates was insufficient.

- **Contribution 1:** a transparent selection policy combining blacklist, cluster, and recency constraints.
- **Contribution 2:** reproducible before/after evidence and a unified regression suite that executes the real inline application script.

2. Related Work

The implementation follows standard browser primitives rather than a learned model. MDN documents `Math.random()` as approximately uniform on $[0, 1)$, supporting integer sampling with `Math.floor` [1]. A JavaScript `Set` stores unique blacklisted names and provides membership checks [2]. Font Awesome documents class-based icon insertion and the `fa-` naming convention used by the page [3]. GitHub Pages is a natural deployment target for this static artifact, although publication was outside the implemented scope [4].

Alternatives considered. A shuffle bag would give cycle-level coverage but complicate blacklist updates; weighted sampling could encode preferences but needs user signals; a backend profile could persist preferences but is unnecessary for a single-session Week 1 app. Explicit client-side filtering was chosen for inspectability and compatibility with the existing no-build structure.

3. Method

Each dish has a name, Font Awesome icon class, and one of six semantic clusters. For recommendation t , the primary eligible set is the catalog minus blacklisted names, the immediately previous cluster, and names in the latest three recommendations. One eligible index is sampled uniformly. If the set becomes empty, the existing fallback relaxes cluster and then cooldown restrictions while preserving the blacklist; this priority rule was intentionally left unchanged.

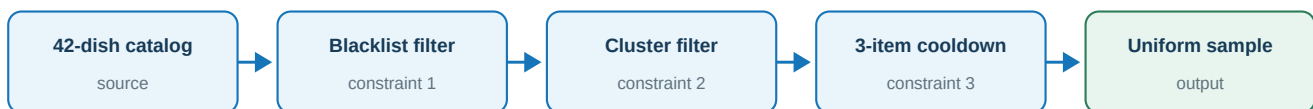


Figure 1. Candidate-generation pipeline. Each stage removes ineligible items before one uniform draw.

Configuration. The catalog contains 42 unique dishes: 6 clusters x 7 dishes. Cooldown length is 3. Blacklist state is stored in memory for the current page session. The remaining-item counter appears only below 10 items. Tests use seeded linear congruential generators (123456789, 246813579, 987654321, and 135792468) and immediate timer completion in a deterministic DOM harness.

Tools. Vanilla HTML5, CSS3, JavaScript; Font Awesome 6.4.0 via a pinned CDN stylesheet; Node.js v25.8.1 for syntax and regression checks; Python 3.12.13 local HTTP server for served-page verification.

4. Experiments

Setup. The evaluation used the static menu as the dataset on a local macOS workstation. Functional tests executed the actual inline JavaScript in a mocked DOM. Success required all deterministic boundaries to match, all dishes to remain reachable, no prohibited transitions in 100,000 draws per behavioral stress test, and zero missing classes in the exact pinned icon stylesheet. Runtime performance was not evaluated.

Measure	Baseline evidence	Final evidence
Unique dishes	12	42
Cluster sizes	No clusters	7 in every cluster
Missing target icons	3	0

Measure	Baseline evidence	Final evidence
Repeat case	Pizza -> Pizza	Pizza -> Sushi
Cooldown boundary	Pizza -> Sushi -> Pizza	Pizza -> Sushi -> Burger -> Salad -> Pizza
Blacklist control	Absent	Present; 0/10,000 blocked-item appearances

Table 1. Baseline-to-final comparison from the saved evidence files.

Stress check	Selections	Violations	Coverage
Adjacent identical dish	100,000	0	42/42 dishes
Adjacent same cluster	100,000	0	42/42 dishes
Three-item cooldown	100,000	0	42/42 dishes
One dish blacklisted	10,000	0 appearances	41/41 alternatives

Table 2. Reproducible final stress tests. The complete suite passed 13/13 checks.

Independent boundary check. In the final sequence `Pizza -> Sushi -> Burger -> Salad -> Pizza`, the two `Pizza` results are separated by exactly three other recommendations. Thus `Pizza` is absent from each of the next three slots after its first appearance and becomes eligible at the required boundary. The balance calculation also checks directly: 42 dishes / 6 clusters = 7 dishes per cluster, matching every parsed cluster count.

5. Discussion

Failure, root cause, and fix

Observed failure: `Ramen: fa-bowl-hot -> MISSING`; `Pasta: fa-pasta -> MISSING`; `Soup: fa-bowl -> MISSING`. **Root cause:** the baseline referenced class names absent from Font Awesome 6.4.0. **Fix:** replace them with `fa-bowl-food`, `fa-plate-wheat`, and `fa-mug-hot`. **Confirmation:** the test fetched the exact pinned stylesheet and found all three classes; the rendered browser check recorded non-empty 64 px glyphs and zero console errors.

What worked

- Filtering an explicit candidate set made blacklist, cluster, and cooldown behavior easy to reason about and test.
- Seeded tests reproduced exact failures before each change and converted them into regression checks afterward.
- Balancing all six clusters removed the catalog-size asymmetry introduced by cluster exclusion.

What failed or was surprising

- Preventing only identical adjacent dishes still allowed near-duplicates such as `Ramen -> Soup`.
- A cluster-only rule still allowed `Pizza` to return after one intervening recommendation.
- Disabled buttons still matched an unguarded CSS hover selector; functional state and visual feedback required separate checks.

Known limitation. When the blacklist leaves too few choices, the existing fallback intentionally relaxes cluster and then cooldown constraints. Therefore blacklist safety is guaranteed, but diversity is not guaranteed in the nearly exhausted state. This behavior was reviewed and intentionally not changed.

Next improvement. Make constraint relaxation visible in the UI (for example, "Repeating a category because only two dishes remain") and add a user setting that chooses between strict diversity and always returning a result.

6. AI Usage Disclosure

Tool used. OpenAI Codex desktop, GPT-5-based coding agent (exact deployed model revision not exposed in the session UI). It was used for repository inspection, implementation, test generation, debugging, official-documentation lookup, evidence organization, and drafting this PDF.

Personal verification. I manually exercised the application during development and identified the visible repeat, same-family repeat, missing-icon, low-variety, and missing-blacklist problems. I checked the cooldown boundary sequence and the $42 / 6 = 7$ cluster calculation shown above. Codex executed the automated suite; the saved output reports 13/13 passing checks, and the served page returned HTTP 200.

Trusted without independent verification. I did not audit the browser engine implementation of `Math.random()`, measure long-run statistical fairness beyond the seeded coverage checks, or benchmark runtime performance. No claim about cryptographic randomness, optimal ranking, or production-scale behavior is made.

Audit trail. The required unmodified, tool-native session export must be submitted separately as `aleksandr_mandrov_a01_session.json`. This report does not create or modify that audit file.

References

- [1] MDN Web Docs, "Math.random()," JavaScript Reference. [Online]. Available: [developer.mozilla.org/.../Math/random](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/random). [Accessed: 2026-09-15].
- [2] MDN Web Docs, "Set," JavaScript Reference. [Online]. Available: [developer.mozilla.org/.../Set](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set). [Accessed: 2026-09-15].
- [3] Fonticons, Inc., "How To Add Icons," Font Awesome Documentation. [Online]. Available: docs.fontawesome.com/web/add-icons/how-to. [Accessed: 2026-09-15].
- [4] GitHub, Inc., "What is GitHub Pages?," GitHub Docs. [Online]. Available: [docs.github.com/.../what-is-github-pages](https://docs.github.com/en/pages/what-is-github-pages). [Accessed: 2026-09-15].

Reproducibility note. Test command: `node assignment1/tests/run-all-tests.js`. Canonical numeric output is stored in `assignment1/tests/latest-results.txt`; per-change before/after artifacts are under `assignment1/evidence/`.

GitHub repository: <https://github.com/MilordEv/RecSys-LLMs/tree/main/assignment1>